



University of
Southern Denmark

EMBEDDED SYSTEMS

DRONE MASTER

1. SEMESTER

GROUP 8

Embedded assignment 2

made by:

Adam Kołodziej: adkol23@student.sdu.dk

Leia P. Villarroel: lepop22@student.sdu.dk

Lucas R. Jeppesen: ljepp19@student.sdu.dk

Oliver Torngaard: oltor19@student.sdu.dk

May 31, 2026

Contents

1	Introduction	3
2	Github	3
3	Frist attempts	3
4	Functionality	5
5	Testing and Validation	8
5.1	HLS	8
5.2	Vitis	8
5.3	UserSpace	9
6	Conclusion	10
7	Appendix	11

List of Figures

1	Project summary	3
2	Memory size error	4
3	Vivado Block Diagram	6
4	DMA IP configuration	7
5	FPGA usage	7
7	Running ROS node camera	13

1 Introduction

The following project consists of storing an image in a reserved memory space using DMA (Direct Memory Access) to stream data to the processor-IP. The Image Processor subscribes to an image topic in ROS, inverts the image with HLS IP, and publishes the processed image. Fig 1 shows a summary of the project:

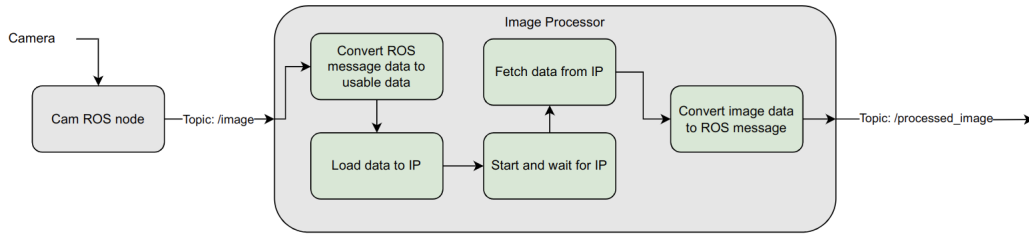


Figure 1: Project summary

The Ultra96-V2 board is used, which is an upgrade of the Ultra96 and it is part of Xilinx Zynq Ultra-Scale+ MPSoC (Multi-Processing System on Chip) to combine HLS (High-Level Synthesis) software with the FPGA (Field-Programmable Gate Array). To connect with the board and integrate the OS (Operating System), Petalinux is used.

2 Github

As it was recommended everyone worked on separate branches and the final work has been merged with the master branch.

Here is the links to GitHub:

FPGA/petaLinux: <https://github.com/lucasrj/embedded-systems-assegment-2>

UserSpace : <https://github.com/lucasrj/embedded-systems-assigment-2-ros>

3 Frist attempts

We were able to choose between different ways of memory implementations:

- Store image directly in processor-IP (Different solution can be explored and defined directly in HLS with #pragmas,

- Store the image in BRAM - implement the read like a dumb logic or implement a AXI-Master that can read from a BRAM-controller,
- Store Image in reserved memory and utilize - a DMA to stream data to the processor-IP or implement AXI-Master in the IP that have direct memory access itself

Due to problems with completion Lecture 4 tasks, we decided to start with the simplest way of implementation - read-like logic. We wanted get familiar with the whole process and understand how everything works. We wrote code in Vitis HLS which is presented below. This code required downscaling the picture coming from the camera. The reason was that we had to be able to squeeze all the data in BRAM of U96V2. We kept an aspect ratio but the resolution of the picture was set to 16x12.

```

1 // Add the vectors
2 int max_pixel_val = 255;
3 int pictureWidth = 16;
4 int pictureLenght = 12;
5 for (int i = 0; i < pictureWidth; i++)
6 {
7     for(int k = 0; k < pictureLenght; k++)
8     {
9         // Insert pragma to utilize FPGA cons
10        #pragma HLS unroll factor = 1000
11        out[i*pictureWidth+k] = max_pixel_val - in[i*pictureWidth+k];
12    }
13 }

```

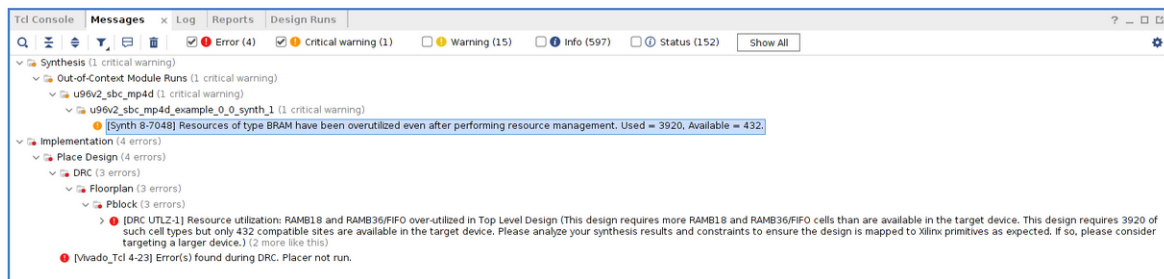


Figure 2: Memory size error

At this point, we weren't sure how we should deal with HLS and Vivado setup so we faced the wall. After conversation with Thor, we decided to change the memory implementation to streaming the data to the processor-IP. So the report focuses on this solution.

4 Functionality

Vitis HLS

When setting up the custom IP block for the image processing, as mentioned earlier the DMA option was chosen. This meant that when creating the IP block, an AXI input stream and output stream were included to work with the data. To work with the AXI streams, "api_axis" was used, which sets up the streams to have the needed element sizes. For this assignment, the only element needed was the data element size, which was set to be 32 bits long for unsigned 32bit int. The user, id, and dest are 0 since the DMA does not use them. To set up the interfaces correctly pragmas hls interface was used. This made it possible to set up the input and output correctly for Vivado. HLS will know which of the ports is the input and output port, which means that the read and write functions can be used freely on the ports. For this project port A was the input while port B was the output. Code inside implements inversions of the picture.

```
1 #include "ap_axi_sdata.h"
2 #include "hls_stream.h"
3 #include <iostream>
4
5 typedef ap_axis<32,0,0,0> mydata_t;
6
7 void example(hls::stream< mydata_t > &A,
8             hls::stream< mydata_t > &B)
9 {
10 #pragma HLS INTERFACE axis port=A
11 #pragma HLS INTERFACE axis port=B
12 #pragma HLS INTERFACE s_axilite port=return bundle=AXI_CPU
13
14 mydata_t tmp;
15
16 do
17 {
18     A.read(tmp);
19     tmp.data = 255 - tmp.data.to_int();
20     B.write(tmp);
21 }while(!tmp.last);
22
23 }
```

Vivado

Unfortunately Github doesn't want to synchronize Vivado files but we will explain what happens there. Similar to what was done in Lecture 7 Direct Memory Access will be implemented in a MPSoC system. In the Vivado block diagram, the high-performance port needs to be enabled in the **Zynq UltraScale+ MPSoC**, and the **AXI Direct Memory Access** block is added to the design. The FPGA usage of the entire design can be seen in figure 7

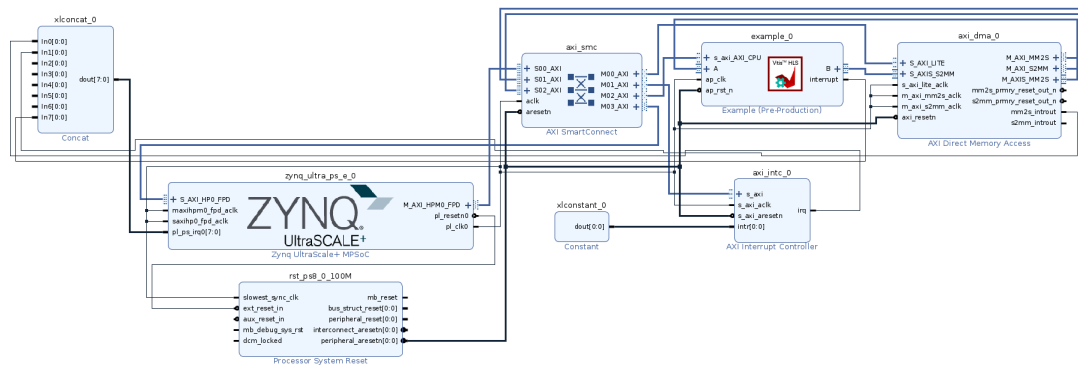


Figure 3: Vivado Block Diagram

DMA IP configuration

The DMA IP is set up like in figure 4. The READ channel allows you to stream data from the memory to the AXI stream interface and the WRITE channel allows you to receive data from the AXI stream interface and write it in the memory. The ports that are labeled mm2s and s2mm refer to memory-mapped to stream and stream to memory-mapped respectively.

The width of the buffer length register is set to 26 so it will be able to transfer 67, 108, 863 bytes which is the maximum size that DMA allows. Ubuntu which is using 64-bit address space so it is required to set the Address Width to 64. Read and Write channels are left enabled, the memory-mapped data is set to 32 and the stream data width is set to 32 bits to match the IP stream width.

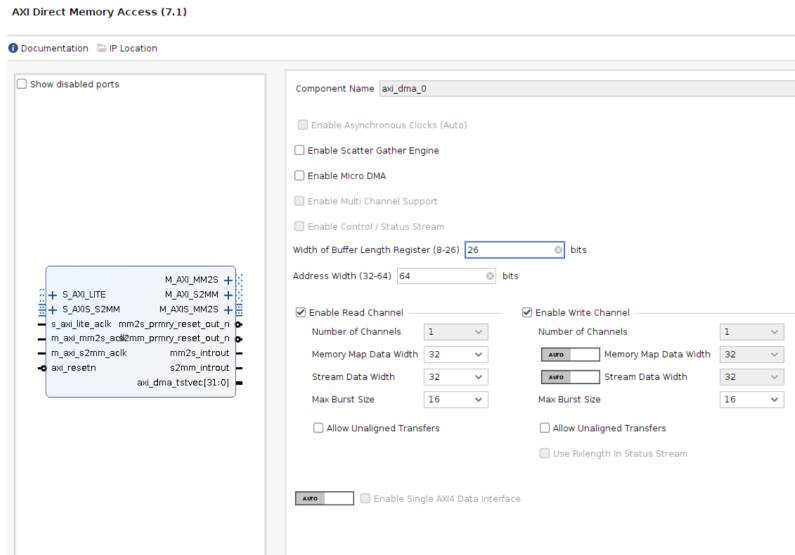


Figure 4: DMA IP configuration

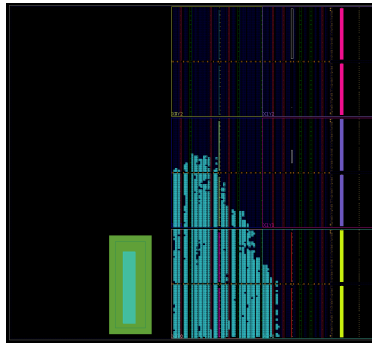


Figure 5: FPGA usage

example_0

We also added the block called "example_0" which was created by us in Vitis HLS. The code inside and how it works is described in the paragraph 4. Connections made are pretty simple. To communicate with UltraSCALE+ AXI protocol is used. As was mentioned before A port is an input and B port is an output. Picture recorded by camera (ROS) is transferred to DMA memory. Then it's going to port A of the example_0 block. Inversions of pixels happens and new version of the picture is sent back to the DMA using B port.

Other connections of this block are rather typical - interrupt, clk, reset.

Vitis

Using Vitis for this project a bare metal test was set up. This was done to test the FPGA setup before the PetaLinux was built. For this project, the file "XExample.h" file was used to set up every bare metal test. When developing the bare metal test for the solution without DMA, a simple function reading, validating, and writing bytes using the in and outputs from the IP block was made. It was done using the XExample read and write_in_r_bytes functions, which are simple read and writes using bytes. If the FPGA correctly inversed every pixel value, the code would show that the test was successful. This bare metal test was sadly not tested, since as mentioned in section 3 we had complications with the solution and changed to the DMA solution.

For the DMA solution, the bare metal test was set up to first check that the FPGA was configured and initiated correctly. The BASE addresses and DMA id was defined using the "xparameters.h" and "xaxidma.h". Thereafter the DMA was flushed in case the cache would be enabled thereby risking the use of non-updated data. Using functions from the "xaxidma.h" and "xexample.h" a data transfer and data validation was set up. This is described in section 5.2, where sudo code of the code is shown.

5 Testing and Validation

For testing and validating the system we test it at 3 stages, on the creation in Vitis HLS to see if it works as expected alone, then in Vitis to see if it works as expected on the FPGA as part of a bigger system, and then lastly in user space to test if it works in the Linux system.

5.1 HLS

For testing it in HLS a test bench was written (see Appendix 7). Stream of data comprised of numbers from 10 to 109 is transferred through port A. Then we run custom "example" block so the data stored in simulated memory is inversed. After this operation we read the data using another stream through port B (output). We should see 245 instead of 10, 244 instead of 11 and so on. The whole process tested by simulation can be observed in the console and in the log file.

Look at the YouTube video: [HLS Simulation](#)

5.2 Vitis

For testing the FPGA diagram in Vitis we have created a program that generates the numbers from 0 to 99 and places them in a buffer in memory, and a receiving buffer is then created, and the pointer to

the buffers is sent to DMA IP together with their size, after which the DMA and custom IP is started, from there the system waits for the IPs to be done, after which the receiving buffer is checked if the data is correct, the test is then repeated 10 times to make sure it is consistent. This can be seen as pseudo-code in Algorithm 1

Algorithm 1 vitis bare-metal test pseudo code

```

1: custom_ip_driver custom_ip()
2: dma_ip_driver dma_ip()
3: uint32_t tx_buffer[32]
4: uint32_t rx_buffer[32]
5: for  $i = 0; i < 32; i++$  do
6:      $tx\_buffer[i] = i$ 
7: end for
8: for  $i = 0; i < 10; i++$  do
9:      $dma\_ip.MM2S(tx\_buffer, size\_of(uint32\_t))$ 
10:     $dma\_ip.S2MM(rx\_buffer, size\_of(uint32\_t))$ 
11:     $custom\_ip.start()$ 
12:     $wait(custom\_ip.done())$ 
13:    for  $i = 0; i < 32; i++$  do
14:        if  $rx\_buffer \neq 255 - tx\_buffer$  then
15:             $print("test failed")$ 
16:             $exit()$ 
17:        end if
18:    end for
19: end for
20:  $print("test success")$ 

```

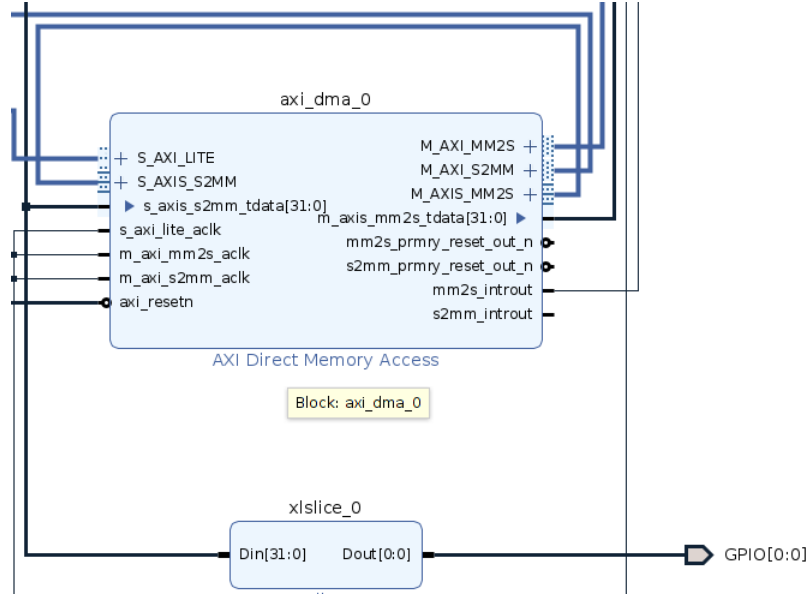
5.3 UserSpace

This part we didn't get working, we tried using the provided DMA example from lecture 7 but it seems to read or write from the wrong location as can be seen in figure 6a, to test if the data is being sent we added an LED and made a vivado design that is setting the led based on the first bit of the stream data(see figure 6b).testing this with Vitis with similar code to Algorithm 1, we could see the LED light up, but with testing it in UserSpace the led do not light up. The code for the UserSpace can be found on GitHub: <https://github.com/lucasrj/embedded-systems-assignment-2-ros>

```

Rx/Tx
131072/0
131074/2
131076/4
131078/6
131080/8
131082/10
131084/12
131086/14
131088/16
131090/18
131092/20
131094/22
131096/24
131098/26
131100/28
131102/30
131104/32
131106/34
131108/36
131110/38
131112/40
131114/42
131116/44
131118/46
131120/48
131122/50
131124/52
131126/54
131128/56
131130/58
131132/60
131134/62
ALL DONE!

```



(a) Output of the UserSpace test

(b) vivado for UserSpace testing

6 Conclusion

From this project, it can be concluded that we did not complete the entire assignment. Every section except the user space ended up working as intended. A complete HLS block for a DMA integration has been created, tested in a test bench, and correctly implemented into a Vivado diagram. From Vivado the FPGA design has been made, with a total FPGA usage of around 25 percent. The hardware setup was tested in Vitis using a bare metal application, where the design was executed correctly without errors. Lastly building a Petalinux kernel and giving it an Ubuntu system to implement the given camera ROS node was done, but we encountered problems with the setup and communication between the userspace and the rest of the setup, making us unable to complete the entire assignment.

7 Appendix

Vitis HLS Testbench

```
1  /*
2  * Copyright 2021 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "License");
5  * you may not use this file except in compliance with the License.
6  * You may obtain a copy of the License at
7  *
8  *   http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing, software
11 * distributed under the License is distributed on an "AS IS" BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 * See the License for the specific language governing permissions and
14 * limitations under the License.
15 */
16
17 #include <iostream>
18 #include "ap_axi_sdata.h"
19 #include "hls_stream.h"
20 using namespace std;
21
22 typedef ap_axis<32,0,0,0> mydata_t;
23
24
25 void example(hls::stream<mydata_t > &A, hls::stream<mydata_t > &B);
26
27 int main()
28 {
29     int i=100;
30     hls::stream<mydata_t > A, B;
31     mydata_t tmp1, tmp2;
32
33     for(int j=0;j<100;j++)
34     {
35         tmp1.data = j+10;
36         tmp1.keep = 1;
37         tmp1.strb = 1;
38         tmp1.user = 1;
39         if(j==99)
40         {
```

```

41     tmp1.last = 1;
42 }
43 else
44 {
45     tmp1.last = 0;
46 }
47 tmp1.id = 0;
48 tmp1.dest = 1;
49 cout << "testing " << tmp1.data << endl;
50 A.write(tmp1);
51 }
52 cout << "data created" << endl;
53     example(A,B);
54 //     B.read(tmp2);
55
56     cout << "Port B data size: " << B.size() << endl;
57
58     while(B.size())
59     {
60         B.read(tmp2);
61         cout << "value after: " << tmp2.data.to_int() << endl;
62     }
63
64 //     cout << tmp2.data.to_int() << endl ;
65
66 //     if (tmp2.data.to_uint() != 0)
67 //     {
68 //         cout << "ERROR: results mismatch" << endl;
69 //         return 1;
70 //     }
71
72
73     cout << "Success: results match" << endl;
74     return 0;
75 }

```

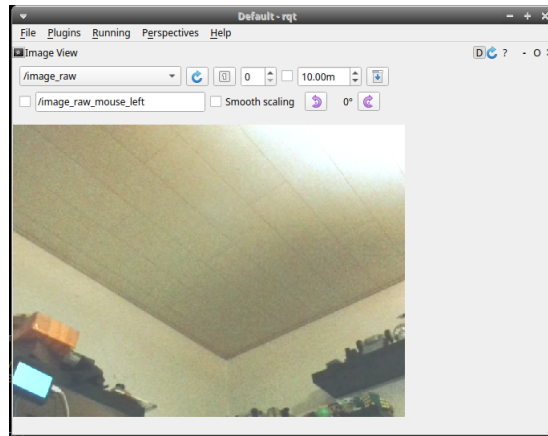


Figure 7: Running ROS node camera